COS 487 / 587 Course Project Part 2: IR Rangers

Esther Greene
University of Southern Maine
Portland, Maine, USA
esther.greene@maine.edu

Courtney Jackson
University of Southern Maine
Portland, Maine, USA
courtney.jackson@maine.edu

Nathaniel Serrano
University of Southern Maine
Portland, Maine, USA
nathaniel.serrano@maine.edu

Abstract

This paper contains our study of retrieving arXiv papers and paragraphs for natural-language research questions, comparing our tuned BM25 baseline with two neural first-stage retrievers: SPLADE, a learned sparse model, and a MiniLM dense bi-encoder. Both are fine-tuned on 1,800 training queries with BM25 hard negatives, index 2.28 million paragraphs, and rank papers by their best paragraph (MaxP). On 250 Study queries, SPLADE is the best single model, raising paper nDCG@5 from 0.514 to 0.631, but on paragraphs its gain over BM25 (0.434 vs. 0.416) is not significant, and the dense model falls below BM25. An ablation shows that much of the neural gain on papers comes from MaxP itself: BM25 ranked by its best paragraph reaches 0.598, statistically indistinguishable from the dense model. Fine-tuning improved validation scores but gave no significant Study gain for either model. Reciprocal rank fusion of all three models is the most effective system, with nDCG@5 of 0.655 on papers and 0.489 on paragraphs. Our analysis shows that the neural models retrieve paraphrased passages that BM25 misses, especially for hard queries, but still fail on shared terms used in another sense and on queries whose meaning lies in mathematical notation.

Keywords

information retrieval, BM25, SPLADE, dense retrieval, rank fusion, scientific document retrieval, passage retrieval

1 Introduction

The volume of scientific literature makes finding relevant work increasingly difficult, both at the level of whole papers and of the specific passages that answer a question. In the first phase of this project, we built a retrieval system for arXiv papers and paragraphs[1] using two term-matching models, and found that BM25 significantly outperformed TF-IDF on both tasks. Our analysis also exposed the limits of exact term matching: hard queries that paraphrase technical terminology in general language, for example asking about "wrist-worn optical sensors" rather than PPG. share few terms with the document that answer them, and BM25 often failed on them, especially for paragraphs.

This paper describes the second phase, in which we add two neural retrieval models intended to bridge this vocabulary gap: a learned sparse model, SPLADE [6], which represents text as weights over vocabulary terms, including related terms the text does not contain, and a dense bi-encoder [13], which represents text as a single embedding vector. We evaluate each model both as released and after fine-tuning on our Train queries, rank papers by their best-scoring paragraph [4], and combine all three models with reciprocal rank fusion [3]. BM25, our best model from part 1, serves as the baseline throughout.

The test collection consists of 34,308 arXiv papers published in August and September 2024. Each paper is provided both as raw HTML scraped from the ar5iv website and as a processed JSON file containing its identifier, title, abstract, and paragraphs. Because the JSON files were produced with rule-based extraction, they contain noise such as leftover LaTeX markup and merged words, and display equations are missing altogether. In total, the collection contains 2,275,574 paragraphs, an average of about 66 per paper.

We address two tasks over this collection: *paper retrieval*, which returns papers identified by their arXiv ID, and *paragraph retrieval*, which returns individual paragraphs. For each query, systems return their top 100 results. Three query sets are provided: Train (2,000 queries), Study (250 queries with relevance judgements, which we use for all analysis in this paper), and Test (250 queries whose judgments are withheld). Each set contains 30% easy, 50% medium, and 20% hard queries. Relevance is graded on a three-point scale: a document that fully addresses the query is relevant (2), one that provides useful information without fully addressing it is partially relevant (1), and any document not in the judgments is considered non-relevant.

SPLADE is the most effective single model on both tasks. On papers it improves nDCG@5 over BM25 from 0.514 to 0.631; on paragraphs its advantage is smaller and not significant. The dense model outperforms BM25 on papers but not on paragraphs, and fine-tuning did not significantly improve either neural model. Fusing all three models gives the best results, with nDCG of 0.701 on papers and 0.550 on paragraphs, significantly above both BM25 and SPLADE alone. Our analysis shows that the neural models can retrieve paragraphs that paraphrase a query without sharing its words, yet they still rank highly some paragraphs that only share its vocabulary.

2 Methodology

Our system retrieves with three first-stage models that share the query sets, run format, and evaluation pipeline. BM25 (Section 2.2) searches inverted indexes over separately preprocessed paper and paragraph collections, as in Part 1. The two neural models, SPLADE (Section 2.4) and a dense bi-encoder (Section 2.5), receive more lightly preprocessed text (Section 2.1) and index paragraphs only. Both start from publicly released checkpoints, which we evaluate as released and after fine-tuning on Train queries (Section 2.3). Because the neural indexes contain only paragraphs, papers are ranked from their paragraphs' scores (Section 2.6). Finally, the runs from different models can be combined with reciprocal rank fusion (Section 2.7). Section 2.8 describes our evaluation setup.

[1] Code: https://github.com/esthergreene/IR-Rangers

2.1 Preprocessing

BM25 and the neural models receive differently preprocessed text, because they match text in different ways.

For BM25, we keep the Part 1 pipeline unchanged. Inline mathematics, remaining LaTeX commands, URLs, and numeric citation markers are removed; text is lowercased and split into alphanumeric tokens; tokens shorter than two characters or consisting only of digits are discarded; stopwords are removed using NLTK's English list [2], extended with terms common in scientific writing such as "et", "al", and "fig"; and the remaining tokens are reduced with the Snowball stemmer [12]. Cleaning reduces the paragraph collection from 168.0 to 114.9 million tokens (31.6%) and leaves 8,389 paragraphs, almost all standalone formulas, empty. These are excluded, leaving 2,267,185 indexed paragraphs.

For the neural models, we remove inline mathematics, LaTeX commands, URLs, and citation markers in the same way, but keep case, stopwords, and word forms, since each encoder applies its own WordPiece tokenizer; queries are cleaned identically. This leaves 2,275,547 paragraphs. Both encoders truncate their input at 256 WordPiece tokens.

2.2 BM25 Baseline

Our baseline is the BM25 model from Part 1, unchanged. It scores a document d for a query q as

$$\text{score}(q,d) = \sum_{t \in q} idf_t \cdot \frac{tf_{t,d}\,(k_1+1)}{tf_{t,d} + k_1\left(1 - b + b\,L_d/L_{avg}\right)}, \tag{1}$$

$$idf_t = \ln\left(1 + \frac{N - df_t + 0.5}{df_t + 0.5}\right), \tag{2}$$

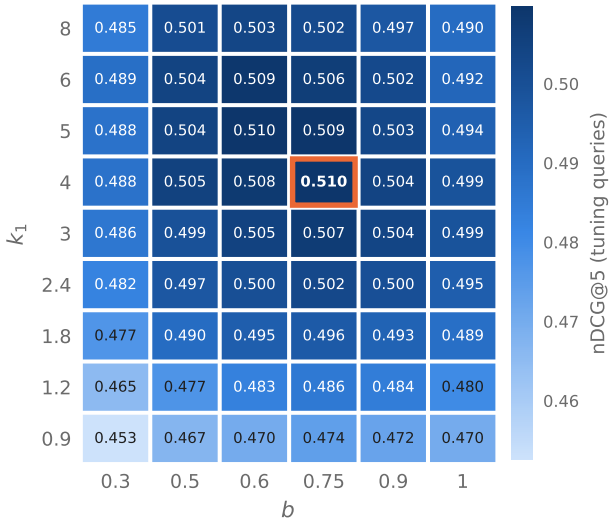
where N is the number of documents, df_t the number containing term t, $tf_{t,d}$ the occurrences of t in d, and L_d and L_{avg} the length of d and the average length, in tokens. This is the "Lucene (accurate)" variant of Kamphuis et al. [9], whose IDF remains positive for terms that occur in more than half of the documents. The parameter k_1 controls how quickly repeated occurrences of a term saturate, and b how strongly scores are normalized by document length. Scoring is term-at-a-time over the postings of the query terms, vectorized with NumPy.

We tuned k_1 and b separately for each collection by grid search on 1,600 Train queries, selecting the setting with the highest nDCG@5 (Figure 1). The selected settings, $k_1 = 4.0$, $b = 0.75$ for papers and $k_1 = 0.4$, $b = 0.9$ for paragraphs, also outperformed both the textbook values ($k_1 = 1.2$, $b = 0.75$) [10] and Anserini's defaults ($k_1 = 0.9$, $b = 0.4$) [15] on the remaining 400 Train queries (nDCG@5 of 0.473 vs. 0.454 and 0.436 on papers, and 0.371 vs. 0.354 and 0.344 on paragraphs). The tenfold difference in k_1 reflects document length: in a paper averaging 3,361 tokens, repeated occurrences of a term indicate its topic, whereas paragraphs, with a median of 40 tokens, rarely repeat a term, so whether a term occurs at all matters more.

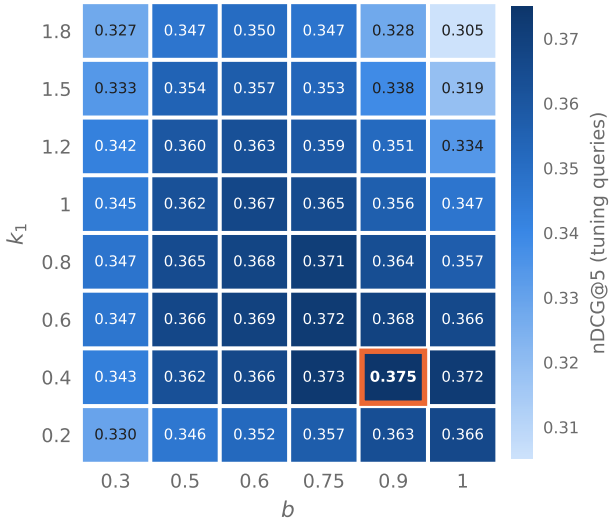
2.3 Training Data

We fine-tune both neural models on Train queries only; Study and Test queries are never used for training or model selection. The 2,000 Train queries are split by query into 1,800 training and 200 validation queries, stratified by difficulty so that both sets keep the 30/50/20 mix of easy, medium, and hard queries, and no validation query appears in training. Every paragraph that was judged relevant or partially relevant serves as a positive, giving 5,138 positives for the training queries and 582 for the validation queries, about 2.9 per query.

The Train judgments contain no non-relevant (grade 0) paragraphs, so negatives must be mined. We use BM25 hard negatives: unjudged paragraphs that BM25 retrieves for the query. They share vocabulary and topics with it and are therefore more informative



(a) Papers.



(b) Paragraphs.

Figure 1: BM25 nDCG@5 on 1,600 Train tuning queries. Outlined cells mark the selected settings.

than random paragraphs. Because judgments are incomplete, an unjudged paragraph near the top of BM25's ranking is often relevant, so we skip BM25's top 20 results and sample 30 negatives per query at random from ranks 21–100, with a fixed seed per query. Random sampling gives each query a mix of close and easier negatives rather than only the hardest ones. Each training example is a (query, positive, negative) triple, one per positive, cycling through the query's negatives. Both neural models are trained with a contrastive loss in which each query's positive must outscore its own hard negative as well as the positives and negatives of the other queries in the batch (Sections 2.4 and 2.5).

Both models select their best checkpoint by nDCG@10 on the validation queries. Encoding all 2.28 million paragraphs at every evaluation would take 30 to 60 minutes (Section 3.7), so validation instead ranks a fixed sample of the collection: every validation positive, the top 100 BM25 results for each validation query, and 10,000 random paragraphs. Scores on this sample are used only to compare checkpoints, not to report effectiveness.

2.4 Learned Sparse Retrieval

This section describes how SPLADE represents text, how we fine-tune it, and how we index and search the paragraph collection with it.

2.4.1 Architecture and Representation. For learned sparse retrieval we use SPLADE [6], initialized from the publicly released checkpoint `naver/splade-cocondenser-selfdistil` [5], which also serves as our zero-shot baseline. SPLADE maps a text (query or paragraph) to a sparse vector $\mathbf{w} \in \mathbb{R}^{|V|}$ over the BERT WordPiece vocabulary ($|V| = 30{,}522$). For each input position t, the Masked Language Modeling (MLM) head produces a logit $h_{t,j}$ for every vocabulary term j; these are passed through a log-saturated ReLU and max-pooled over positions (padding masked out):

$$w_j = \max_t \log\left(1 + \mathrm{ReLU}(h_{t,j})\right) \tag{3}$$

The ReLU zeroes out most terms, max-pooling keeps each term's strongest contribution anywhere in the text, and the logarithm damps very large weights so that no single term dominates a score. Because the MLM head scores the whole vocabulary, a text can receive weight on terms it does not contain, which lets SPLADE expand queries and documents with related terms while still matching exact terms. Inputs are truncated at 256 tokens.

2.4.2 Model Fine-Tuning. We fine-tune the base model on the training triples described in Section 2.3 with `SpladeLoss`, which combines `SparseMultipleNegativesRankingLoss` with FLOPS regularization [11]. The ranking loss is a softmax cross-entropy in which each query's positive paragraph must outscore its BM25 hard negative and all other paragraphs in the batch. A no-duplicates batch sampler keeps a query from appearing twice in one batch, so its other positives are never treated as negatives. FLOPS regularization penalizes the sum over the vocabulary of each term's squared mean weight across the batch, which discourages terms that are active in many texts and so keeps posting lists short; we weight it 5×10^{-5} for queries and 3×10^{-5} for documents.

Fine-tuning settings:

- **Batch size:** 32 (mixed precision, fp16)
- **Learning rate:** 1×10^{-5}
- **Training horizon:** up to 3 epochs
- **Validation and early stopping:** validation nDCG@10 every 50 steps; training stops after 3 evaluations without improvement, and the best checkpoint is kept.

The best checkpoint (step 300, epoch 1.86) raised validation nDCG@10 from 0.566 to 0.644.

2.4.3 Indexing and Retrieval.

Index construction. Cleaned paragraphs are encoded on the GPU in batches. Each batch's activations are copied to host memory in a single transfer and stored as a small CSR matrix. The batches are then stacked into one paragraph–vocabulary matrix and converted to Compressed Sparse Column (CSC) format, so that each vocabulary term's column, holding the paragraphs that contain it and their weights, is contiguous, as in an inverted index. The matrix is saved as a compressed `.npz` file, together with the paragraph IDs in row order and the encoder settings. Retrieval loads this saved index; it is rebuilt only on request.

Search. Queries are encoded by the same pipeline into $\mathbf{q} \in \mathbb{R}^{|V|}$, and a paragraph's score is the sparse dot product

$$S(q, d) = \langle \mathbf{q}, \mathbf{d} \rangle = \sum_{j \in V} q_j \, d_j \tag{4}$$

Only the columns of the query's active terms are read, which works like term-at-a-time scoring over an inverted index. The top k paragraphs are returned with their scores; paragraphs that share no active term with the query score zero and are left out.

2.5 Dense Retrieval

For dense retrieval, we used the pretrained `all-MiniLM-L6-v2` Sentence-BERT bi-encoder [13, 14]. The model encodes queries and paragraphs into 384-dimensional dense embeddings, so it retrieves documents based on semantic similarity rather than exact term overlap. Embeddings are normalized, so inner-product similarity is equivalent to cosine similarity.

The model was fine-tuned on the 5,138 training triples (Section 2.3) with MultipleNegativesRankingLoss, in which each query's positive paragraph must outscore its BM25 hard negative and every other paragraph in the batch. Training used a batch size of 32, a learning rate of 2×10^{-5}, and up to five epochs; validation nDCG@10 was computed every 50 steps, training stopped after three evaluations without improvement, and the best checkpoint (step 250, epoch 1.55) was kept.

We indexed 2,275,547 paragraph embeddings with FAISS [8] using `IndexFlatIP`, which performs exact inner-product search. Each query is encoded with the same bi-encoder, and the index returns the highest-scoring paragraphs.

2.6 Paper Ranking

Our neural models index paragraphs only, so paper rankings are built from paragraph rankings with MaxP [4]: each paper is scored by its best-scoring retrieved paragraph,

$$\mathrm{score}(p) = \max_{d \in p \cap R_q} s(q, d), \tag{5}$$

where $s(q, d)$ is the model's score for paragraph d and R_q is the set of paragraphs retrieved for query q. We retrieve the top 2,000 paragraphs, map each to its paper, and walk down the list keeping the first occurrence of each paper; since the list is sorted by score, a paper's first occurrence is its best paragraph. We stop at 100 distinct papers. Retrieval goes far deeper than 100 because one paper often has several paragraphs near the top of the ranking; at a depth of 2,000, every Study query yielded 100 distinct papers. Papers with no paragraph in the top 2,000 are not ranked. The BM25 baseline keeps its own full-paper index; in Section 4.2 we also rank papers with BM25 by MaxP to separate the effect of MaxP from that of neural matching.

2.7 Rank Fusion

BM25, SPLADE, and the dense model match queries to documents in different ways, through exact terms, weighted and expanded terms, and a single embedding, so their errors may differ as well. To test whether they complement each other, we combine their rankings with reciprocal rank fusion (RRF) [3], which scores each document by its rank in each run:

$$\mathrm{RRF}(d) = \sum_{m \in M} \frac{1}{k + \mathrm{rank}_m(d)}, \qquad k = 60, \tag{6}$$

where M is the set of fused runs and $\mathrm{rank}_m(d)$ the rank of d in run m; a document absent from a run contributes nothing for that run. RRF uses only ranks, so it needs no normalization of the models' differently scaled scores, and the constant k limits how much a single top rank can dominate. We use the standard $k = 60$ without tuning. We fuse the top-100 runs of BM25 and SPLADE, and of all three models, separately for each collection, and keep the top 100 fused results per query. For papers, the BM25 paper run is fused with the neural models' MaxP paper runs. Fusion is evaluated on Study queries only and is not among our submitted Test runs.

2.8 Evaluation Setup

We evaluate each run's top 100 results with six metrics: nDCG, nDCG@5, MRR, P@5, P@10, and Recall@100, computed with ranx [1]. For a ranking with relevance grades $rel_i \in \{0, 1, 2\}$ at ranks $i = 1, 2, \ldots,$

$$\mathrm{DCG@}k = \sum_{i=1}^{k} \frac{rel_i}{\log_2(i+1)}, \qquad \mathrm{nDCG@}k = \frac{\mathrm{DCG@}k}{\mathrm{IDCG@}k}, \tag{7}$$

where IDCG@k is the DCG of an ideal ranking of the judged documents [7]; nDCG without a cutoff uses the full top 100. MRR is the mean over queries of $1/r$, where r is the rank of the first relevant document (0 if none is retrieved), P@k is the fraction of the top k documents that are relevant, and Recall@100 is the fraction of a query's relevant documents that appear in the top 100. Recall@100 is new in this phase: it measures how much a model could contribute to fusion or to a later reranking stage, beyond its top ranks. Documents absent from the judgments count as non-relevant.

nDCG uses the relevance grades directly as gains. For MRR, P@k, and Recall@100, documents of grade 1 or 2 count as relevant. We also report strict MRR and P@5, in which only grade-2 documents count as relevant; these are averaged over the 238 Study queries that have at least one grade-2 document.

We test differences between runs with a two-sided paired Student's t-test over queries at $p < 0.05$, using ranx, between every pair of runs in a table: BM25, SPLADE, and the dense model; the zero-shot and fine-tuned version of each neural model; and each fused run against BM25, SPLADE, and the other fused run. We do not correct for multiple comparisons, so with many tests some individual significant differences may be by chance; we therefore emphasize differences that are consistent across metrics and collections.

The three query sets serve distinct purposes. Train queries are used for tuning BM25 (Section 2.2) and for training the neural models as well as selecting their checkpoints (Section 2.3). The Study queries are used for all reported results, and the Test queries, whose judgments are withheld, are for our submitted runs and manual annotation (Section 3.6).

3 Evaluation Results

This section reports how the neural models trained (Section 3.1), their effectiveness on the Study queries, including the effects of fine-tuning, rank fusion, and query difficulty (Sections 3.2–3.5), and each model's cost (Section 3.7).

3.1 Training

Figure 2 shows the training curves. For SPLADE, validation loss fell sharply in the first third of an epoch and reached its minimum at step 300 (epoch 1.86), where validation nDCG@10 also peaked at 0.644, up from 0.566 for the zero-shot model; early stopping ended training at about epoch 2.8. Fine-tuning also made SPLADE less sparse: the average number of active terms rose from 103 to 203 per query and from 184 to 363 per paragraph.

3.2 Effectiveness

Table 1 compares the three models. SPLADE is the most effective single model on both tasks. On papers, both neural models significantly outperform BM25 on nDCG, nDCG@5, MRR, P@5, and both strict metrics, with SPLADE raising nDCG@5 from 0.514 to 0.631, and SPLADE is also significantly better than Dense on nDCG@5, P@5, and P@10. On paragraphs the gains are much smaller: SPLADE scores higher than BM25 on every metric, but significantly only on Recall@100 (0.630 vs. 0.592) and strict P@5, while Dense falls slightly below BM25 on nDCG (0.447 vs. 0.474), with no significant difference between the two. SPLADE is significantly better than Dense on every paragraph metric except P@5 and Recall@100. Section 4.1 discusses why the neural models gain more on papers than on paragraphs, and Section 4.2 separates the effect of MaxP.

3.3 Effect of Fine-Tuning

As shown in Table 2, fine-tuning did not lead to significant improvements for either neural model. For Dense retrieval, paper-level nDCG increased from 0.635 to 0.640, while paragraph-level nDCG decreased from 0.458 to 0.447. The zero-shot Dense model also performed significantly better on paper-level P@5 (0.166 vs. 0.156), and P@10 (0.090 vs. 0.084; $p < 0.05$). This suggests that fine-tuning did not consistently improve Dense retrieval on our Study queries. Although paper-level nDCG increased slightly, paragraph-level nDCG and paper-level P@5 and P@10 decreased.

Table 1: Effectiveness on the 250 Study queries. Strict metrics count only grade-2 documents (238 queries). Superscripts mark a significant improvement over the model with that letter (paired t-test, $p < 0.05$).

	Model	nDCG	nDCG@5	MRR	P@5	P@10	R@100	Strict MRR	Strict P@5
Papers									
a	BM25	0.582	0.514	0.544	0.137	0.080	0.803	0.535	0.124
b	Dense	0.640^{a}	0.592^{a}	0.631^{a}	0.156^{a}	0.084	0.807	0.606^{a}	0.141^{a}
c	SPLADE	**0.671**^{a}	**0.631**^{ab}	**0.658**^{a}	**0.167**^{ab}	**0.090**^{ab}	**0.825**	**0.645**^{a}	**0.151**^{ab}
Paragraphs									
a	BM25	0.474	0.416	0.516	0.174	0.107	0.592	0.450	0.115
b	Dense	0.447	0.377	0.477	0.175	0.104	0.607	0.415	0.115
c	SPLADE	**0.494**^{b}	**0.434**^{b}	**0.539**^{b}	**0.190**	**0.116**^{b}	**0.630**^{a}	**0.484**^{b}	**0.135**^{ab}

Table 2: Zero-shot versus fine-tuned neural models on Study queries. † marks a significant difference between the two versions of a model (paired t-test, $p < 0.05$).

	Papers					Paragraphs				
Model	nDCG	nDCG@5	MRR	P@5	R@100	nDCG	nDCG@5	MRR	P@5	R@100
Dense, zero-shot	0.635	0.591	0.619	0.166†	0.812	0.458	0.385	0.484	0.174	0.633
Dense, fine-tuned	0.640	0.592	0.631	0.156	0.807	0.447	0.377	0.477	0.175	0.607
SPLADE, zero-shot	0.668	0.625	0.654	0.161	0.815	0.485	0.431	0.524	0.186	0.603
SPLADE, fine-tuned	0.671	0.631	0.658	0.167	0.825	0.494	0.434	0.539	0.190	0.630

SPLADE showed small improvements after fine-tuning, with paragraph-level nDCG increasing from 0.485 to 0.494. None of these differences were statistically significant. Overall, our results suggest that fine-tuning did not provide a clear advantage over the pretrained models on the Study query set.

3.4 Rank Fusion

Table 3 shows that fusion helps most on paragraphs. Fusing BM25 with SPLADE significantly improves over both on paragraph nDCG (0.531, vs. 0.494 for SPLADE) and nDCG@5, but on papers it is no better than SPLADE alone. Adding the dense model gives the best results on every metric for both collections, with paper nDCG of 0.701 and paragraph nDCG of 0.550. On paragraphs, RRF of all three models is significantly better than BM25, SPLADE, and RRF(BM25+SPLADE) on nearly every metric, even though the dense model alone is the weakest paragraph model. On papers, its gains over SPLADE are significant for nDCG, P@10, and Recall@100, but not for nDCG@5 or MRR. We used the standard $k = 60$ without tuning, and fusion is not among our submitted Test runs.

3.5 Query Difficulty

Table 4 breaks nDCG@5 down by query difficulty. On papers, the neural models improve most on easy and medium queries: SPLADE raises nDCG@5 over BM25 by 0.155 on easy and 0.133 on medium queries, but by only 0.023 on hard queries, where neither model differs significantly. It is only with fusion that we see a significant difference in nDCG (0.276 RRF vs. 0.187 BM25).

On paragraphs, BM25 remains the best single model on easy queries (0.676), significantly ahead of the dense model (0.559) and numerically ahead of SPLADE (0.627). Easy queries tend to reuse the wording of their source paragraph, which exact term matching rewards. SPLADE leads on medium queries (0.475 vs. 0.425 for BM25, not significant), and fusion is significantly better than every single model (0.547). On hard paragraph queries, all models score close to zero: 0.003 for BM25 and about 0.04 for the neural models. The dense model is significantly better than BM25, but no model reliably places a judged relevant paragraph in its top five. Incomplete judgments may contribute to this: in Part 1, BM25's top-ranked paragraph was unjudged for every hard Study query. Our Test annotations (Section 3.6) give a fuller picture of how the models perform on hard queries.

3.6 Test Annotation

For each of the 250 Test queries we judged the top-ranked paragraph of our best Study model, fine-tuned SPLADE, using the same 0–2 scale and guidelines as in Part 1: *relevant* (2), *partially relevant* (1), or *non-relevant* (0). Annotators saw the paragraph's original text, including equations, and the paper's title. For 79 queries (32%), SPLADE's top paragraph was the one BM25 had ranked first in Part 1, so we reused its Part 1 grade; the remaining 171 were split evenly among the three of us (57 each), balanced by difficulty.

SPLADE's top paragraph is at least partially relevant for 203 queries (81%), against 197 (79%) for BM25, and fully relevant for 113 (45%), against 104 (42%) (Table 5). The gain comes from hard queries, where SPLADE's top paragraph is relevant for 32 of 50 against 22 for BM25, while BM25 stays slightly ahead on easy queries (70 vs. 66 of 75), the same pattern as in the Study results (Section 3.5). These counts are descriptive: each model has one judged paragraph per query, and the two sets of judgments were made in different phases of the project.

Table 3: Reciprocal rank fusion on Study queries. Superscripts mark a significant improvement over the run with that letter (paired t-test, $p < 0.05$).

	Run	nDCG	nDCG@5	MRR	P@5	P@10	R@100	Strict MRR	Strict P@5
Papers									
a	BM25	0.582	0.514	0.544	0.137	0.080	0.803	0.535	0.124
b	SPLADE	0.671^{a}	0.631^{a}	0.658^{a}	0.167^{a}	0.090^{a}	0.825	0.645^{a}	0.151^{a}
c	RRF(BM25+SPLADE)	0.675^{a}	0.628^{a}	0.659^{a}	0.170^{a}	0.094^{a}	0.834^{a}	0.646^{a}	0.149^{a}
d	RRF(all three)	$\mathbf{0.701}^{abc}$	$\mathbf{0.655}^{ac}$	$\mathbf{0.689}^{ac}$	$\mathbf{0.174}^{a}$	$\mathbf{0.095}^{ab}$	$\mathbf{0.853}^{abc}$	$\mathbf{0.673}^{ac}$	$\mathbf{0.154}^{a}$
Paragraphs									
a	BM25	0.474	0.416	0.516	0.174	0.107	0.592	0.450	0.115
b	SPLADE	0.494	0.434	0.539	0.190	0.116	0.630^{a}	0.484	0.135^{a}
c	RRF(BM25+SPLADE)	0.531^{ab}	0.468^{ab}	0.569^{a}	0.201^{a}	0.119^{a}	0.666^{ab}	0.508^{a}	0.136^{a}
d	RRF(all three)	$\mathbf{0.550}^{abc}$	$\mathbf{0.489}^{abc}$	$\mathbf{0.582}^{ab}$	$\mathbf{0.218}^{abc}$	$\mathbf{0.130}^{abc}$	$\mathbf{0.689}^{abc}$	$\mathbf{0.523}^{ab}$	$\mathbf{0.148}^{abc}$

Table 4: nDCG@5 on Study queries by difficulty (75 easy, 125 medium, 50 hard). Superscripts mark a significant improvement over the model with that letter within the same group (paired t-test, $p < 0.05$).

		BM25 (a)	Dense (b)	SPLADE (c)	RRF (d)
Papers	Easy	0.635	0.771^{a}	0.790^{a}	$\mathbf{0.804}^{a}$
	Medium	0.571	0.639	0.704^{ab}	$\mathbf{0.717}^{ab}$
	Hard	0.187	0.207	0.210	$\mathbf{0.276}^{a}$
Paragraphs	Easy	0.676^{b}	0.559	0.627^{b}	$\mathbf{0.694}^{bc}$
	Medium	0.425	0.404	0.475^{b}	$\mathbf{0.547}^{abc}$
	Hard	0.003	0.036^{a}	**0.041**	0.036^{a}

Table 5: Grades of the top-ranked paragraph for the 250 Test queries, by difficulty: fine-tuned SPLADE (Part 2) and BM25 (Part 1). For 79 queries both models ranked the same paragraph first, and its Part 1 grade was reused.

Grade	Easy	Medium	Hard	All
SPLADE (Part 2)				
Relevant (2)	47	54	12	113
Partially relevant (1)	19	51	20	90
Non-relevant (0)	9	20	18	47
Relevant or partially	88%	84%	64%	81%
BM25 (Part 1)				
Relevant (2)	49	48	7	104
Partially relevant (1)	21	57	15	93
Non-relevant (0)	5	20	28	53
Relevant or partially	93%	84%	44%	79%

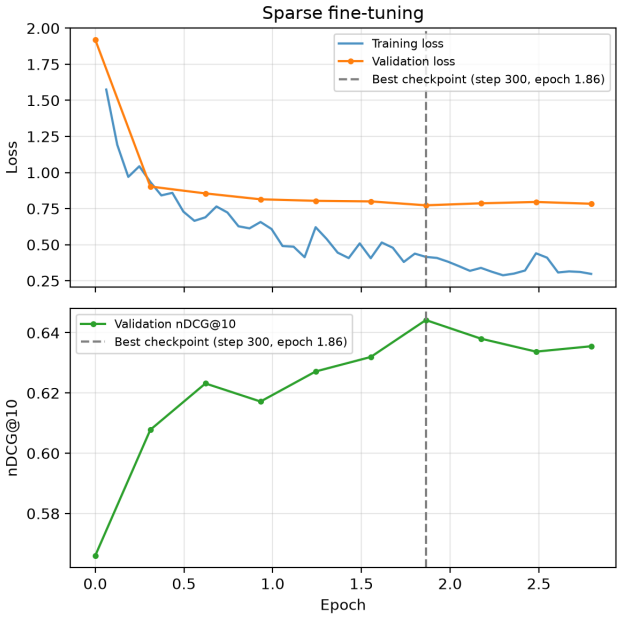
3.7 Efficiency

BM25 is by far the cheapest model (Table 6): it builds its paragraph index in under three minutes on the CPU and answers a paragraph query in 62 ms, against 230 ms for the dense model and 491 ms for SPLADE. Neural paper retrieval costs about as much as paragraph retrieval, since it is a 2,000-paragraph search followed by MaxP (Section 2.6). For the dense model the two latencies are identical, because exact search compares the query with all 2.28 million paragraph vectors whatever the depth; this brute-force CPU search, not query encoding, accounts for most of its latency, and an approximate or GPU index would reduce it. SPLADE is the slowest model even though it searches only the columns of its query terms: fine-tuned queries activate about 200 terms, many of them common expansion terms whose columns cover a large share of the collection, and our implementation uses no pruning. Its weaker sparsity after fine-tuning (Section 3.1) adds to this cost.

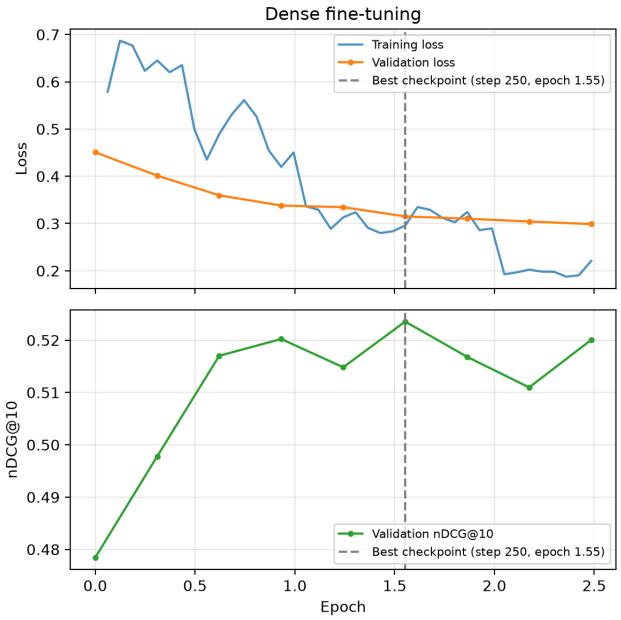
Table 6: On-disk index size, indexing time, startup time and average query latency on the 250 Study queries (papers / paragraphs). All times were measured on one shared server with an Intel Core i7-11700 CPU (2.50 GHz), 31 GiB of RAM and an NVIDIA RTX A5000 GPU (24 GB); encoders run on the GPU and index search on the CPU. Query latency includes query processing and, for neural paper runs, retrieving 2,000 paragraphs and MaxP. Neural models index only paragraphs. With a checkpoint every 50,000 paragraphs, dense indexing took 70.1 minutes.

Model	Index	Indexing	Startup (s)	Query (ms)
BM25	242 MB / 974 MB	81 s / 175 s	15.6 / 34.6	7.3 / 62.4
Dense	3.3 GB	31.8 min	64.5 / 41.0	231.6 / 230.1
SPLADE	2.8 GB	63.4 min	97.3 / 69.1	536.5 / 490.9

The neural indexes are also three to four times larger than BM25's paragraph index. The dense index stores 384 floats per paragraph (3.3 GB), and the SPLADE index holds 812 million nonzero weights, about 357 per paragraph, which take 2.8 GB compressed on disk and about 6.5 GB in memory. Encoding the collection on the GPU took 31.8 minutes for MiniLM and 63.4 minutes for SPLADE, whose BERT-base encoder is larger and scores the full vocabulary at every token position. Startup is longer for neural paper runs than for paragraph runs because they also load the paragraph-to-paper map. Fusing all three models would cost the sum of their latencies, about 0.8 s per query if run one after another. Finally, all times come from single runs on a shared server and should be read as approximate.



(a) SPLADE.



(b) Dense (MiniLM).

Figure 2: Training and validation loss and validation nDCG@10 for each model; the dashed line marks the selected (best) checkpoint.

4 Results Analysis

We now examine why the models perform as they do: how SPLADE and the dense model differ, what the neural models gain over BM25 and where that gain comes from, example queries on which they succeed and fail, and why fine-tuning and fusion had the effects they did. We close with a summary of each system's strengths and weaknesses.

Table 7: Paper retrieval on Study queries when BM25 also ranks papers by their best paragraph (BM25-MaxP), as the neural models do. The last three columns give nDCG@5 by difficulty. Superscripts mark a significant improvement over the model with that letter (paired t-test, $p < 0.05$).

	Model	nDCG	nDCG@5	P@5	R@100	nDCG@5 Easy	nDCG@5 Med.	nDCG@5 Hard
a	BM25	0.582	0.514	0.137	0.803	0.635	0.571	0.187
b	BM25-MaxP	0.645^{a}	0.598^{a}	0.152^{a}	0.788	**0.830**^{a}	0.642	0.141
c	Dense	0.640^{a}	0.592^{a}	0.156^{a}	0.807	0.771^{a}	0.639	0.207
d	SPLADE	**0.671**^{a}	**0.631**^{ac}	**0.167**^{abc}	**0.825**	0.790^{a}	**0.704**^{abc}	**0.210**

4.1 Comparing the Neural Models

As we see in Table 1, SPLADE performed better than Dense retrieval on both papers and paragraphs. For paper retrieval, SPLADE was able to achieve an nDCG of 0.671 compared to 0.640 for Dense. For paragraph retrieval, SPLADE also performed better, with an nDCG of 0.494 compared to 0.447. The differences in paragraph-level nDCG were statistically significant.

One possible explanation is how the two models represent text. SPLADE uses vocabulary-based representations that can preserve important technical terms while also expanding beyond exact word matches. Dense retrieval represents each paragraph using a 384-dimensional embedding, which helps capture semantic similarity but may lose some specific technical information.

Dense retrieval also outperformed the original BM25 baseline on papers, but not on paragraphs. This improvement may come from the MaxP paper-ranking approach rather than from dense retrieval itself: as shown in Table 7, BM25 with MaxP achieved a paper-level nDCG of 0.645 compared to 0.640 for Dense, with no statistically significant difference between the models. Since BM25-MaxP performed similarly to Dense, we cannot attribute the improvement to the Dense model alone.

4.2 Comparison with the BM25 Baseline

Comparing our Part 2 models with the Part 1 baseline on papers conflates two changes: the neural models use a different matching model, and they also rank papers differently, by their best paragraph rather than by the paper's full text. To separate the two, we also ranked papers with BM25 by MaxP, retrieving 2,000 paragraphs with the paragraph-tuned BM25 (Section 2.2) and keeping each paper's best paragraph (Table 7). This alone raises BM25's paper nDCG from 0.582 to 0.645. In fact, it significantly increases on every metric except P@10 and Recall@100. BM25-MaxP is statistically indistinguishable from the dense model on every metric. The dense model's advantage over BM25 on papers therefore comes from ranking papers by their best paragraph, not from dense matching, which is consistent with its weaker paragraph results. SPLADE remains ahead of BM25-MaxP (nDCG 0.671 vs. 0.645), but the difference is significant only for P@5; its advantage is concentrated on medium queries (nDCG@5 0.704 vs. 0.642, significant). With every model

utilizing the same paper ranking method, SPLADE's improvement over BM25 becomes modest on both tasks, and the large gains in this phase come from MaxP and from fusion (Section 3.4).

MaxP helps most on easy queries, where BM25-MaxP reaches nDCG@5 of 0.830, the highest of any model. Many queries ask about a specific finding or method described in a single passage, so a paper's best-matching paragraph is a better indicator of relevance than its full text, in which a long paper can accumulate matches for common query terms across many unrelated paragraphs, as seen with query 2112 in Part 1. On hard queries the opposite holds: BM25-MaxP is the weakest model (0.141), and full-paper BM25 is significantly better on nDCG (0.256 vs. 0.201). When a query paraphrases its source passage, no single paragraph shares many terms with it, but the paper as a whole may.

Query 2011, which BM25 failed in Part 1, shows where neural matching does help: *"Why is it harder to identify specific heart rhythm abnormalities using wrist-worn optical sensors than with electrical cardiac monitoring, and what complicates this process in wearable devices?"* Its relevant paper describes this in technical terms ("arrhythmias", "PPG", "ECG", "smartwatch"), and none of its five judged paragraphs appear in BM25's top 100. Both neural models retrieve them. The dense model ranks three partially relevant paragraphs, including the abstract, 3rd, 4th, and 6th, and SPLADE ranks the fully relevant paragraph 8th and a partially relevant one 9th. On papers, the relevant paper rises from 12th with BM25 to 3rd with the dense model and 5th with SPLADE. The two models also succeed differently: the dense model places the fully relevant paragraph only 55th, while SPLADE places it 8th but ranks the abstract only 54th. Such disagreements are what fusion can exploit, since it rewards documents that several models rank well. Query 2011 is a hard query, so it also shows that the near-zero hard-query averages in Table 4 hide individual successes of this kind.

4.3 Successful Retrieval Cases

In order to properly understand where neural models succeed compared to the BM25 baseline, we investigate some example queries from the collection. Test query 2500 shows how SPLADE bridges vocabulary mismatch: *"What factors regarding the heterogeneity of clients influence the utility of regulating order flows in logistics services?"* SPLADE ranks first a paragraph we judged fully relevant: *"Insight 4: The potential gains of demand management increase with the level of heterogeneity between the customer types…"* The paragraph shares only one word with the query, *heterogeneity*; it says *demand management* for regulating order flows, *gains* for utility, and *customer types* for clients. BM25 does not rank it in its top 100, since a paragraph matching a single query term is outranked by many that match several, and the dense model ranks it only 22nd. SPLADE combines an exact match on the rare term *heterogeneity* with weight on related terms for the rest of the query, which neither exact term matching nor a single dense vector manages here.

The same pattern holds across our Test annotations: on hard queries, which paraphrase their source passage most, SPLADE's top paragraph is relevant for 32 of 50 queries against 22 for BM25 (Table 5). Query 2011 (Section 4.2) gives a Study example in which both neural models retrieve paragraphs that BM25 misses entirely.

4.4 Failure Cases

SPLADE's top paragraph was judged non-relevant for 47 of the 250 Test queries (Section 3.6). Four patterns account for most of them.

Shared terms in another sense. Like BM25 in Part 1, SPLADE still ranks paragraphs that share a distinctive query term used in a different sense. Query 2433 (*"What is the purpose of utilizing a decay parameter for epsilon during a search procedure?"*) asks about decaying the exploration rate, but SPLADE's top paragraph is from a particle-physics paper about the decay channels of long-lived particles. Query 2391 asks for a technique that combines penalized regression with resampling to select variables, and SPLADE returns a paragraph on the resampling step of particle filters. Expansion does not resolve such ambiguity, since it adds terms related to the word rather than to the query's intended sense.

Topical drift. Expansion can also cause errors that pure term matching cannot. For query 2264, about *"abrupt climatic shifts and solar-driven occurrences"*, SPLADE's top paragraph shares no content word with the query: it comes from an autonomous-driving paper about robustness to adverse weather, a related topic but not the query's subject.

Lost notation. Because our cleaning removes LaTeX math from queries as well as paragraphs, queries about specific mathematical objects lose their most specific terms. Query 2400 (*"How is the function $\Omega_4^{(4)}$ expressed in terms of ω for the variables $\lambda_1, \ldots, \lambda_4$?"*) is reduced to generic words such as *function*, *expressed*, and *variables*. SPLADE's top paragraph is non-relevant for 8 of the 16 Test queries that contain LaTeX, against 39 of the 234 that do not (50% vs. 17%), although the former sample is small. Keeping math in a normalized form, or encoding it separately, is a clear direction for future work.

Incomplete judgments. Some apparent failures are not failures. About 89% of each model's top 10 Study paragraphs are unjudged (88.4% for SPLADE, 89.3% for BM25, 89.6% for the dense model), and since the Study qrels list almost only relevant paragraphs, every unjudged paragraph counts as non-relevant. Because the rate is nearly the same for all three models, this lowers absolute scores without favoring any model. Our Test annotations suggest the effect is large: SPLADE's top paragraph was at least partially relevant for 81% of Test queries in our judgment, whereas its Study MRR of 0.539 means its first-ranked paragraph is judged relevant for at most 54% of Study queries. The query sets and annotators differ, so the comparison is only indicative, but our Study scores are best read as lower bounds.

4.5 Why Fine-Tuning Gave Little

One possible reason fine-tuning did not improve our results significantly is that both pretrained models had already learned useful representations from large training datasets. Our training set contained around 1,800 queries, which may not have been enough to make a noticeable improvement over the original models. This could be an explanation of why we saw only small changes in retrieval performance after fine-tuning.

Another possible factor is how the training data was prepared. Both models were trained against hard negatives selected by BM25,

so they learned to separate relevant paragraphs from BM25-style distractors rather than from their own mistakes.

Also, our relevance judgements were limited, meaning some paragraphs treated as negative examples may have actually been relevant. We attempted to reduce this risk during BM25 negative sampling by skipping the top 20 results. Finally, selecting checkpoints using validation nDCG@10 from a relatively small query set may have contributed to differences between validation performance and our final Study results.

4.6 Why Fusion Helps

Our results suggest that combining different retrieval models can improve performance because each model approaches retrieval differently. BM25 focuses on exact term matching, SPLADE combines term matching with learned expansion, and Dense retrieval uses semantic similarity. Because of these differences, the models may retrieve relevant documents that the others miss.

As we see in Table 3, combining all three models using Reciprocal Rank Fusion (RRF) produced the best overall results. Paper-level nDCG increased from 0.671 for SPLADE alone to 0.701 for RRF, while paragraph-level nDCG increased from 0.494 to 0.550.

Something interesting to note is that adding Dense to the BM25 and SPLADE combination improved paragraph-level nDCG from 0.531 to 0.550, even though Dense had the lowest individual paragraph-level nDCG. This suggests that Dense contributed useful retrieval results that complemented the other two models.

4.7 Strengths and Weaknesses

Each retrieval approach had different strengths and weaknesses. BM25 was useful for matching exact terms, including technical names and acronyms. It was also relatively simple and efficient, but struggled when queries used different terminology from relevant documents.

SPLADE was our strongest individual retrieval model. By combining vocabulary-based matching with learned term expansion, it could retrieve relevant documents even when the query and document did not use exactly the same words. However, it still had limitations when similar terms appeared in unrelated scientific contexts. It also required more computational resources than BM25.

Dense retrieval was useful for capturing semantic similarity between queries and documents, but it performed worse than SPLADE overall. Its compact embeddings may have lost some important technical details, particularly when retrieving specific paragraphs.

Finally, RRF achieved the best overall performance by combining the rankings from all three models. This approach also required running multiple retrieval systems, increasing computational cost compared to using a single model.

5 Conclusion

We extended our BM25 system with learned sparse and dense retrieval over arXiv paragraphs. SPLADE was the most effective single model, and both neural models significantly outperformed BM25 on paper retrieval, but an ablation showed that ranking papers by their best paragraph accounts for much of that gain; on paragraphs, neural matching alone improved little over a well-tuned BM25. Fine-tuning on our training queries improved validation effectiveness but did not carry over to the full collection, and it reduced SPLADE's sparsity, increasing its index size and query latency. Because the three models fail on different queries, fusing them with RRF gave the best results on both tasks, at the cost of running all three retrievers.

Several directions follow from our analysis. Mining hard negatives from the models' own rankings over the full collection, or distilling from a cross-encoder, would train the models on their actual errors rather than BM25's. A cross-encoder could rerank the fused top 100. Preserving mathematical notation in a normalized form would help the queries that our cleaning currently strips of their most specific terms. Finally, tuning the fusion constant or learning fusion weights, and trying encoders pretrained on scientific text, may improve effectiveness further.

References

[1] Elias Bassani. 2022. ranx: A blazing-fast python library for ranking evaluation and comparison. In *European Conference on Information Retrieval*. Springer, 259–264.

[2] Steven Bird, Ewan Klein, and Edward Loper. 2009. *Natural Language Processing with Python*. O'Reilly Media.

[3] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Büttcher. 2009. Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*. 758–759.

[4] Zhuyun Dai and Jamie Callan. 2019. Deeper Text Understanding for IR with Contextual Neural Language Modeling. In *Proceedings of the 42nd International ACM SIGIR Conference on Research and Development in Information Retrieval*. 985–988.

[5] Thibault Formal, Carlos Lassance, Benjamin Piwowarski, and Stéphane Clinchant. 2022. From Distillation to Hard Negative Sampling: Making Sparse Neural IR Models More Effective. In *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2353–2359.

[6] Thibault Formal, Benjamin Piwowarski, and Stéphane Clinchant. 2021. SPLADE: Sparse Lexical and Expansion Model for First Stage Ranking. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2288–2292.

[7] Kalervo Järvelin and Jaana Kekäläinen. 2002. Cumulated Gain-Based Evaluation of IR Techniques. *ACM Transactions on Information Systems* 20, 4 (2002), 422–446.

[8] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2021. Billion-Scale Similarity Search with GPUs. *IEEE Transactions on Big Data* 7, 3 (2021), 535–547.

[9] Chris Kamphuis, Arjen P. de Vries, Leonid Boytsov, and Jimmy Lin. 2020. Which BM25 Do You Mean? A Large-Scale Reproducibility Study of Scoring Variants. In *Advances in Information Retrieval: 42nd European Conference on IR Research (ECIR 2020)*. Springer, 28–34.

[10] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. 2008. *Introduction to Information Retrieval*. Cambridge University Press. https://nlp.stanford.edu/IR-book/

[11] Biswajit Paria, Chih-Kuan Yeh, Ian E. H. Yen, Ning Xu, Pradeep Ravikumar, and Barnabás Póczos. 2020. Minimizing FLOPs to Learn Efficient Sparse Representations. In *International Conference on Learning Representations*.

[12] Martin F. Porter. 2001. Snowball: A Language for Stemming Algorithms. https://snowballstem.org/texts/introduction.html.

[13] Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*. 3982–3992.

[14] Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. 2020. MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers. In *Advances in Neural Information Processing Systems*, Vol. 33.

[15] Peilin Yang, Hui Fang, and Jimmy Lin. 2017. Anserini: Enabling the use of lucene for information retrieval research. In *Proceedings of the 40th international ACM SIGIR conference on research and development in information retrieval*. 1253–1256.